

Rust Program Security Audit Report

Protocol: SRCOIN TOKEN (Strategic Reserve Coin)
Chain: Solana
Framework: Anchor v0.31.0 with Token-2022 Extensions (TransferHook, TransferFeeConfig, NonTransferable)
Audit Date: March 14, 2026
Audited By: Raju Raj Kishore
Program IDs:

- `src_token` : CmStJVyiTg6h6DpWexjD4hYmKMCF9VdCFegw7agAT2id
- `transfer_hook` : F2fLCTN6UQBQFTBPHGARKqkkQjzKzm9AuoeXSzqv2lre
- `swap_pool` : AjZtMppVtPX5ZBCy7N2br8uRhvsre3eXu2GfucMTNRmS
- `sol_pool` : 5axkDAEFUpkWg5FupExtZymVMlWgYba4NMsDEhZHiCJz

Scope

File	Lines	Description
programs/src_token/src/lib.rs	907	Core token issuance, DAO multisig, vesting, NFT governance
programs/transfer_hook/src/lib.rs	562	Token-2022 transfer hook, fee collection, burn mechanism
programs/swap_pool/src/lib.rs	579	FUNLESS → SRCOIN fixed-rate swap pool (migration)
programs/sol_pool/src/lib.rs	853	SOL ↔ SRCOIN constant-product AMM
SRCOIN WP - FINAL.pdf	12 pages	Whitepaper cross-reference
Total	2,901 LOC	

Excluded:

- `tests/` – test infrastructure
- `target/` – build artifacts
- `node_modules/` – client dependencies

Executive Summary

SRCOIN TOKEN is a four-program Solana ecosystem implementing a community-governed memecoin with Bitcoin-inspired reserve discipline. It uses Token-2022 extensions (TransferHook for per-transfer logic, TransferFeeConfig for 1.29% fee withholding, and NonTransferable for soulbound governance NFTs), a FUNLESS-to-SRCOIN migration swap pool, a SOL/SRC constant-product AMM, and a time-plus-price-gated vesting system with DAO multisig governance.

The overall security posture is **weak**. The audit identified **101 valid findings**: 1 Critical, 8 High, 22 Medium, 17 Low, and 38 Informational across the four programs. An additional **16 whitepaper-to-code discrepancy findings** were identified: 2 Critical, 4 High, 4 Medium, 4 Low, and 2 Informational.

The most critical systemic pattern is **permissionless initialization** – every program's initialization instruction can be front-run by any caller, permanently capturing token supply distribution, fee governance, vesting schedules, and pool ownership. The second systemic pattern is **unconstrained mint parameters** – fake mint attacks are possible across `swap_pool( book_swap )`, `sol_pool` (all swap/liquidity functions), and `src_token( unlock_funds , claim_nft )`, allowing attackers to generate real entitlements from worthless tokens. The whitepaper cross-reference revealed that the NFT governance system implements only 2 of 3 documented tiers, at 10x lower thresholds, with no token locking mechanism.

Account Interaction Map

Program Accounts

Account	Type	Authority	Program	Purpose
PoolConfig	Account<PoolConfig>	admin (Signer)	swap_pool	Swap pool configuration and state
Booking	Account<Booking>	user PDA	swap_pool	Per-user swap booking record
SolPoolConfig	Account<SolPoolConfig>	admin (Signer)	sol_pool	AMM pool configuration and reserves
SolVault	Account<SolVault>	pool_authority PDA	sol_pool	SOL reserve vault
DaoTreasuryMultisig	Account<DaoTreasuryMultisig>	dao-multisig PDA	src_token	DAO signer list and threshold

Account	Type	Authority	Program	Purpose
UnlockManager	Account<UnlockManager>	unlock-manager PDA	src_token	Vesting plan state
NftClaim	Account<NftClaim>	nft-claim PDA	src_token	Per-user-per-tier NFT claim record
NftRegistry	Account<NftRegistry>	nft-registry PDA	src_token	Global NFT claim enumeration
CrankConfig	Account<CrankConfig>	crank-config PDA	transfer_hook	Authorized crank wallet
BurnLimiter	Account<BurnLimiter>	burn-limiter PDA	transfer_hook	Yearly and lifetime burn caps
ExtraAccountMetaList	Raw PDA	extra-account-metas PDA	transfer_hook	Transfer hook extra accounts

External CPI Calls

Instruction	CPI Target	Authority	Trust
add_liquidity, claim (swap_pool)	SPL Token-2022 transfer_checked	pool_authority PDA	⚠ Missing transfer hook accounts
swap_sol_for_src, swap_src_for_sol	SPL Token-2022 transfer_checked	pool_authority PDA	⚠ Missing transfer hook accounts
unlock_funds, dao_transfer	SPL Token-2022 transfer_checked + transfer hook	unlock_authority / dao_authority PDA	✅ Hook accounts resolved
claim_nft	SPL Token-2022 mint, freeze, set_authority	nft_mint_authority PDA	✅ Verified
harvest_fees, collect_fees	SPL Token-2022 transfer_fee instructions	fee_authority PDA	✅ Verified

Value Flow

User FUNLESS → book_swap() → funless_vault → withdraw_funless() → multisig src_coin_vault → claim() → User SRC (after 90-day lock)

User SOL → swap_sol_for_src() → sol_vault, src_coin_vault → User SRC

User SRC → swap_src_for_sol() → src_coin_vault, sol_vault → User SOL

unlock_vault → unlock_funds() → destination (DAO multisig approved)

dao_treasury → dao_transfer() → destination (DAO multisig approved)

Transfer fees → harvest_fees() → mint withheld → collect_fees() → 40% burn + 60% crank

Findings

ID	Title	Severity	Confidence	Status
S-01	Fake Mint Attack – Unconstrained Funless Mint Generates Real SRC Entitlements	Critical	High	Open
S-02	Protocol Paralysis – Missing Token-2022 Transfer Hook Extra Accounts on All SRC Transfers	High	High	Open
S-03	Permanent State Desynchronization – pool_config Missing mut in remove_liquidity	High	High	Open
S-04	Admin-User Race Condition – Rug Pull at Lock Expiry	High	High	Open
S-05	Vault Substitution – Missing has_one on srcoin_vault and funless_vault	High	High	Open
R-01	No Total Supply Cap Validation – Caller Distributes Arbitrary Token Amounts	High	High	Open
R-02	Permissionless initialize_token_with_hook – First Caller Captures Supply	High	High	Open
R-03	Permissionless initialize_unlock_manager – First Caller Sets Vesting	High	High	Open
R-04	Caller-Supplied released Field Initializes Plans with Phantom Released Tokens	High	High	Open
R-05	start_ts = 0 Not Validated – Complete Time-Vesting Bypass	High	High	Open
R-06	NFT Registry Exhaustion – 100-Entry Cap Permanently Disables Claims	High	High	Open
T-01	Permissionless initialize_crank_config – First Caller Hijacks Fee Governance	High	High	Open
T-02	Permissionless initialize_extra_account_meta_list – Unconstrained Mint	High	High	Open
P-01	Protocol Paralysis – Missing Transfer Hook Extra Accounts on All SRC Transfers	High	High	Open
P-02	Unconstrained srcoin_mint – Fake Mint Attack Across Entire Program	High	High	Open

ID	Title	Severity	Confidence	Status
P-03	Vault Substitution – Missing has_one on srcoin_vault and sol_vault	High	High	Open
S-06	Governance Deadlock – Duplicate Signers in initialize_pool	Medium	High	Open
S-07	Phantom Liquidity – Transfer Fee Ignored in add_liquidity	Medium	High	Open
S-08	Exchange Rate Corruption – Decimal Normalization Missing in book_swap	Medium	High	Open
S-09	Silent Fee Deduction on Claims – User Receives Less Than Booked	Medium	High	Open
T-03	amount * 129 Arithmetic Overflow – Transfer DoS Above ~143M Tokens	Medium	High	Open
T-04	Pubkey::default() Accepted as authorized_crank	Medium	High	Open
T-05	No On-Chain DAO Revenue Enforcement – 60% Fee Share Trusts Crank Script	Medium	High	Open
T-06	Arithmetic Overflow in collect_fees Fee Split Calculations	Medium	High	Open
T-07	Unconstrained mint in harvest_fees and collect_fees	Medium	High	Open
R-07	No Duplicate Signer Validation in initialize_token_with_hook	Medium	High	Open
R-08	No Duplicate Signer Validation in dao_update_signers	Medium	High	Open
R-09	Atomic Full Signer Replacement With No Timelock	Medium	High	Open
R-10	Duplicate Category Names Create Unreachable Vesting Plans	Medium	High	Open
R-11	Missing Plan Count Limit and Category String Length Validation	Medium	High	Open
R-12	Unconstrained mint in unlock_funds – Fake Mint Exhausts Budget	Medium	High	Open
R-13	Unconstrained src_mint in claim_nft – Forgery Bypasses Eligibility	Medium	High	Open
R-14	Price Escalation Bypass via Batch Claiming	Medium	High	Open
R-15	per_period = 0 Freezes Price Escalation for Small Allocations	Medium	Medium	Open

ID	Title	Severity	Confidence	Status
R-16	Sybil Attack on NFT Gating via Flash Loan	Medium	Medium	Open
P-04	Slippage Protection Ineffective – Transfer Fee Applied After Check	Medium	High	Open
P-05	CEI Violations in Swap and Liquidity Functions	Medium	High	Open
P-06	Token-2022 Transfer Fee Shortfall in Remove Liquidity	Medium	High	Open
S-10	CEI Violations – State Updated After CPIs	Low	High	Open
T-08	dao_treasury Permanently Baked into ExtraAccountMetaList	Low	High	Open
T-09	Silent u64 Truncation – as u64 Narrowing Casts	Low	High	Open
T-10	DAO Fee Share Left in Crank Wallet After collect_fees	Low	High	Open
R-17	unlock_vault Has No Canonical Address Binding	Low	High	Open
R-18	dao_treasury Has No Canonical Address Binding	Low	High	Open
R-19	Self-Transfer in dao_transfer – Treasury Converts to Fees	Low	High	Open
R-20	dao_authority and unlock_authority Empty CHECK With No Seeds	Low	High	Open
R-21	Systemic Missing token::mint Constraints	Low	High	Open
R-23	Unbounded period_count – CU Exhaustion	Low	High	Open
P-07	Silent u64 Truncation in SRC Lamport Scaling	Low	High	Open
P-08	Lock Floor Incompatibility – 82-Year Lock vs Swap Depletion	Low	Medium	Open
P-09	Rounding Inconsistency Between quote_swap and Execution	Low	High	Open
P-10	Misleading Error Code and Dead Guard in compute_output	Low	High	Open
P-11	Volume Counter checked_add – Protocol Bricks at u64::MAX	Low	High	Open
S-11	lock_end_ts Overflow is DoS via Panic	Informational	High	Open
S-12	booked_at Timestamp Overwritten on Repeat Bookings	Informational	High	Open

ID	Title	Severity	Confidence	Status
S-13	Claims Allowed During Pause	Informational	High	Open
S-14	is_active Guard Missing on Admin Operations	Informational	Medium	Open
S-15	Centralization Risk – Single Admin Key	Informational	High	Open
T-11	Permissionless initialize_burn_limiter	Informational	High	Open
T-13	fallback Lacks Token-2022 Caller Validation	Informational	Medium	Open
T-14	associated_token_program Unused	Informational	High	Open
T-15	Fee Estimates Are Approximations	Informational	High	Open
T-16	Silent Mode Change When Burn Cap Is Hit	Informational	High	Open
T-17	No Reserved Space in State Structs	Informational	High	Open
R-22	multiplier_bps < 10000 Creates Deflating Schedule	Informational	High	Open
R-24	secondary_price_multiplier_bps = 0 Collapses Price Gate	Informational	High	Open
R-25	unlock_manager.released Updated After CPI	Informational	High	Open
R-26	decimals Caller-Supplied Instead of Derived	Informational	High	Open
R-27	No Minimum Amount Guard in unlock_funds and dao_transfer	Informational	High	Open
R-28	dao_treasury_multisig Marked mut Unnecessarily	Informational	High	Open
R-29	system_program and rent Unused in DaoTransfer	Informational	High	Open
R-30	No On-Chain Accounting of DAO Treasury Outflows	Informational	High	Open
R-31	Freeze Authority Never Revoked – NFTs Confiscatable	Informational	High	Open
R-32	Permissionless init_nft_registry	Informational	High	Open
R-33	Plain Addition in Log Line Causes Overflow Panic	Informational	High	Open
R-34	No Reserved Bytes in State Structs	Informational	High	Open
R-35	CustomError Enum Missing Required Variants	Informational	High	Open
R-36	rent Sysvar Unnecessary in Multiple Structs	Informational	High	Open
P-12	CEI Violation and Misleading Log in initialize_pool	Informational	High	Open

ID	Title	Severity	Confidence	Status
P-13	Access Control Asymmetry – Admin vs Multisig	Informational	High	Open
P-14	set_pool_active Unilateral Admin Pause	Informational	High	Open
P-15	DEFAULT Constants Never Enforced	Informational	High	Open
P-16	Dead Liquidity – Pre-Existing Vault Balance Ignored	Informational	High	Open
P-17	No Structured Event Emission	Informational	High	Open
—————	Whitepaper Discrepancy Findings	—————	—————	—————
WP-01	NFT Tier Thresholds 10x Lower – Satoshi Guard Missing	Critical	High	Open
WP-02	Tier 3 (Voting Member) Not Implemented	Critical	High	Open
WP-03	Transaction Fee 1.29% vs Documented 1%	High	High	Open
WP-04	One Wallet = One Tier Not Enforced	High	High	Open
WP-05	All NFTs Non-Transferable (Tier 2/3 Should Transfer)	High	High	Open
WP-06	No SRCOIN Lock Mechanism for Governance	High	High	Open
WP-07	Per-Tier NFT Caps Not Enforced – Global 100 vs 2,775	Medium	High	Open
WP-08	Undisclosed 40% Fee Burn	Medium	High	Open
WP-09	Allocations Runtime-Configurable, Not Hardcoded	Medium	High	Open
WP-10	Total Supply Cap Not Validated On-Chain	Medium	High	Open
WP-11	Milestone Evaluation Dates Not Enforced	Low	High	Open
WP-12	Phase One Binary Release/Burn Not Enforced	Low	High	Open
WP-13	Phase Two Annual Burn Path Does Not Exist	Low	High	Open
WP-14	150M/8M/842M Allocation Split Unchecked	Low	High	Open
WP-15	On-Chain Governance Voting Not Implemented	Informational	High	Open
WP-16	Tier 3 Annual 1,000 SRCOIN Reward Missing	Informational	High	Open

Finding Details

S-01 – Fake Mint Attack – Unconstrained Funless Mint Generates Real SRC Entitlements

Severity: Critical

Confidence: High

File: `programs/swap_pool/src/lib.rs`

Lines: 184–256, 438–470

Chain: Solana (Anchor)

Vector Category: Unconstrained Mint / Account Substitution

Description

The `book_swap` instruction accepts a `funless_mint` account that is only constrained to match the `funless_vault`'s mint (`token::mint = funless_mint` at line 446). However, the `funless_vault` is not bound to `pool_config.funless_mint` via `has_one`. An attacker can create a worthless token mint, create a vault owned by `pool_authority` holding that fake mint, and pass it as the `funless_vault` along with their fake `funless_mint`. The booking PDA is derived from `pool_config.key()` and `user.key()` – not from the mint – so the booking records real SRC entitlements against a worthless deposit.

Impact

An attacker can book unlimited SRC entitlements by depositing worthless fake tokens. After the 90-day lock expires, the attacker calls `claim` to withdraw real SRC from the `src_coin_vault`. The entire SRC liquidity in the pool can be drained at zero cost.

Exploit Scenario

1. Attacker creates a new SPL token mint (`fake_funless_mint`) with 1 trillion supply
2. Attacker creates a token account for `fake_funless_mint` with authority `pool_authority` PDA
3. Attacker deposits fake tokens into this vault
4. Attacker calls `book_swap` with `funless_mint = fake_funless_mint` and `funless_vault = fake_vault`
5. `token::mint = funless_mint` constraint passes (vault holds the fake mint)
6. `pool_config` records `total_src_coin_booked` against real SRC deposits
7. After 90 days, attacker calls `claim` with the real `src_coin_vault` and receives real SRC
8. Pool drained completely

Proof of Concept

```

#[tokio::test]
async fn exploit_fake_mint_book_swap() {
    // Setup: Initialize pool with real funless/srcoin mints
    // Deposit 150M SRC liquidity

    // Attack: Create fake funless mint
    let fake_mint = Keypair::new();
    create_mint(&mut banks, &fake_mint, &attacker.pubkey(), 9).await;

    // Create fake vault owned by pool_authority PDA
    let fake_vault = create_token_account(
        &mut banks, &fake_mint.pubkey(), &pool_authority_pda
    ).await;

    // Mint 1B fake tokens to attacker
    mint_to(&mut banks, &fake_mint, &attacker_ata, 1_000_000_000e9 as u64)

    // Book swap with fake mint – gets real SRC entitlement
    let ix = swap_pool::instruction::book_swap(
        500_000_000e9 as u64, // 500M fake funless
    );
    // Pass fake_mint and fake_vault instead of real ones
    // srcoin_amount = 500M * 0.3 = 150M SRC booked against real vault

    // Wait 90 days, then claim real SRC
    warp_to(pool.lock_end_ts + 1);
    let claim_ix = swap_pool::instruction::claim();
    // Passes: booking.srcointitled = 150M, srcoin_vault has 150M
    // Attacker receives 150M real SRC for 0 cost
}

```

Recommended Fix

Add `has_one = funless_mint` and `has_one = funless_vault` constraints to the `BookSwap` account struct:

Before:

```

pub struct BookSwap<'info> {
    #[account(mut, seeds = [b"pool-config"], bump)]
    pub pool_config: Account<'info, PoolConfig>,
    pub funless_mint: InterfaceAccount<'info, Mint>,
    #[account(mut, token::mint = funless_mint, token::authority = pool_auth)]
    pub funless_vault: InterfaceAccount<'info, TokenAccount>,
}

```

After:

```
pub struct BookSwap<'info> {  
    #[account(mut, seeds = [b"pool-config"], bump, has_one = funless_mint,  
    pub pool_config: Account<'info, PoolConfig>,  
    pub funless_mint: InterfaceAccount<'info, Mint>,  
    #[account(mut, token::mint = funless_mint, token::authority = pool_auth  
    pub funless_vault: InterfaceAccount<'info, TokenAccount>,  
}
```

S-02 – Protocol Paralysis – Missing Token-2022 Transfer Hook Extra Accounts

Severity: High

Confidence: High

File: `programs/swap_pool/src/lib.rs`

Lines: 64–76, 116–129, 159–172, 281–294

Chain: Solana (Anchor)

Vector Category: Token-2022 Transfer Hook Incompatibility

Description

All SRC token transfers in `swap_pool` use `token::transfer_checked` via Anchor's CPI helper without providing the extra accounts required by the Token-2022 TransferHook extension. The SRC mint has a TransferHook extension pointing to program

`F2fLCTN6UQBQFTBPHGARKqkkQjzKzm9AuoeXSzqv21re`, which requires an `ExtraAccountMetaList` PDA, a `dao_treasury` account, and a `burn_limiter` PDA. Without these accounts, every `transfer_checked` CPI involving SRC tokens will fail at runtime with `AccountNotFound`.

Impact

The `add_liquidity`, `remove_liquidity`, and `claim` instructions are permanently non-functional. Admins cannot deposit SRC into the pool, cannot withdraw SRC liquidity, and users cannot claim their booked SRC entitlements after the lock period. The swap pool is effectively bricked for all SRC operations.

Exploit Scenario

1. Admin deploys `swap_pool` and initializes with SRC and FUNLESS mints
2. Admin calls `add_liquidity` to deposit SRC into the pool vault

3. Token-2022 runtime invokes `transfer_checked` which checks for TransferHook extension
4. TransferHook requires extra accounts from ExtraAccountMetaList PDA
5. Swap pool CPI does not include these accounts
6. Transaction fails with `AccountNotFound` or `MissingRequiredSignature`
7. No SRC can ever enter or leave the swap pool

Recommended Fix

Replace `token::transfer_checked` with manual

`spl_token_2022::instruction::transfer_checked` + `add_extra_accounts_for_execute_cpi` as done in `src_token`'s `unlock_funds` and `dao_transfer` instructions. Add `extra_account_meta_list`, `transfer_hook_program`, `dao_treasury`, and `burn_limiter` accounts to all instruction structs that transfer SRC.

S-03 – Permanent State Desynchronization – `pool_config` Missing `mut`

Severity: High

Confidence: High

File: `programs/swap_pool/src/lib.rs`

Lines: 387-388

Chain: Solana (Anchor)

Vector Category: State Mutation Failure

Description

The `RemoveLiquidity` account struct declares `pool_config` without the `mut` attribute at line 387. Line 132 writes `pool.total_srcoin_deposited = pool.total_srcoin_deposited.saturating_sub(amount)`, but since the account is not marked mutable, Anchor will not serialize the updated state back to the account. The `total_srcoin_deposited` counter remains at its pre-removal value permanently.

Impact

After any `remove_liquidity` call, `pool_config.total_srcoin_deposited` is permanently inflated relative to actual vault balance. The overbooking check in `book_swap` (line 206: `pool_after_booked <= pool.total_srcoin_deposited`) uses this stale value, allowing bookings that exceed actual SRC in the vault. When users later claim, the vault has insufficient balance and claims fail silently or the last claimers lose their tokens.

Recommended Fix

```
// Before
#[account(seeds = [b"pool-config"], bump)]
pub pool_config: Account<'info, PoolConfig>,

// After
#[account(mut, seeds = [b"pool-config"], bump)]
pub pool_config: Account<'info, PoolConfig>,
```

S-04 – Admin-User Race Condition – Rug Pull at Lock Expiry

Severity: High

Confidence: High

File: `programs/swap_pool/src/lib.rs`

Lines: 97-138, 259-306

Chain: Solana(Anchor)

Vector Category: Race Condition / Rug Pull

Description

Both `remove_liquidity` (admin multisig) and `claim` (user) become callable at exactly `clock.unix_timestamp >= pool.lock_end_ts`. There is no ordering guarantee. An admin multisig can front-run all user claims by calling `remove_liquidity` for the entire vault balance in the same slot that the lock expires. Users who deposited FUNLESS and waited 90 days for their SRC entitlements find the vault empty.

Impact

The admin multisig can extract all SRC liquidity at lock expiry before any user can claim, effectively stealing all user entitlements. Users lose their booked SRC and their deposited FUNLESS (already transferred to the `funless_vault` and withdrawable by admin via `withdraw_funless`).

Recommended Fix

Add a claim-first window: modify `remove_liquidity` to require `clock.unix_timestamp >= pool.lock_end_ts + CLAIM_WINDOW_SECONDS` (e.g., 30 days), giving users time to claim before admin can remove liquidity. Alternatively, enforce `pool.total_srcoin_deposited - amount >= pool.total_srcoin_booked - pool.total_srcoin_claimed` to prevent removing liquidity that backs unclaimed bookings.

S-05 – Vault Substitution – Missing `has_one` on `srcoin_vault` and `funless_vault`

Severity: High

Confidence: High

File: `programs/swap_pool/src/lib.rs`

Lines: 473-504, 411-435

Chain: Solana (Anchor)

Vector Category: Account Substitution

Description

The `Claim` struct (lines 473-504) does not bind `srcoin_vault` to `pool_config.srcoin_vault` via `has_one`. The `WithdrawFunless` struct (lines 411-435) does not bind `funless_vault` to `pool_config.funless_vault` via `has_one`. An attacker can pass any token account with matching mint and authority as the vault, redirecting tokens to an account they control.

Impact

For `claim`: An attacker who is also a legitimate claimant can substitute a different SRC token account as `srcoin_vault`, causing the pool to send SRC from a different source while the real vault balance remains unchanged. For `withdraw_funless`: Admin can redirect FUNLESS withdrawals to an unrelated vault.

Recommended Fix

Add `has_one = srcoin_vault` to `Claim` and `has_one = funless_vault` to `WithdrawFunless`:

```
#[account(mut, seeds = [b"pool-config"], bump, has_one = srcoin_vault)]
pub pool_config: Account<'info, PoolConfig>
```

S-06 – Governance Deadlock – Duplicate Signers in `initialize_pool`

Severity: Medium

Confidence: High

File: `programs/swap_pool/src/lib.rs`

Lines: 25-54

Chain: Solana (Anchor)

Vector Category: Governance / Duplicate Validation

Description

The `initialize_pool` function accepts `admins: Vec<Pubkey>` at line 28 without checking for duplicate entries. An initializer can pass `[keyA, keyA, keyA]` with `admin_threshold = 3`. The multisig checks in `remove_liquidity` and `withdraw_funless` deduplicate via `seen.contains()`, so `keyA` signing counts as only 1 approval out of the required 3. The pool is permanently deadlocked – no admin operation requiring multisig consensus can ever succeed.

Impact

If initialized with duplicates, `remove_liquidity` and `withdraw_funless` are permanently locked. SRC liquidity and collected FUNLESS are trapped in the vaults forever. The pool cannot be redeployed because the PDA is deterministic.

Recommended Fix

Add duplicate detection:

```
let mut unique_admins = admins.clone();
unique_admins.sort();
unique_admins.dedup();
require!(unique_admins.len() == admins.len(), SwapPoolError::DuplicateAdmins);
```

S-07 – Phantom Liquidity – Transfer Fee Ignored in add_liquidity

Severity: Medium

Confidence: High

File: `programs/swap_pool/src/lib.rs`

Lines: 57-82

Chain: Solana (Anchor)

Vector Category: Token-2022 Transfer Fee Misaccounting

Description

The `add_liquidity` function records `pool.total_srcoin_deposited += amount` at line 79, where `amount` is the gross transfer amount. SRC has a 1.29% `TransferFeeConfig`, so the vault actually receives `amount * (1 - 0.0129)`. The deposited counter is inflated by ~1.29% relative to actual vault balance. Over multiple deposits, the gap widens.

Impact

The overbooking check `pool_after_booked <= pool.total_srcoin_deposited` allows bookings up to the inflated counter, which exceeds actual vault balance. Users who book near the limit will find insufficient SRC when claiming.

Recommended Fix

Use the pattern from `sol_pool : record balance_before`, transfer, reload vault, compute `actually_received = balance_after - balance_before`, and update the counter with `actually_received`.

S-08 – Exchange Rate Corruption – Decimal Normalization Missing

Severity: Medium

Confidence: High

File: `programs/swap_pool/src/lib.rs`

Lines: 193–197

Chain: Solana (Anchor)

Vector Category: Decimal Mismatch

Description

The swap rate calculation `srcoin_amount = funless_amount * 3 / 10` at lines 193–197 assumes both FUNLESS and SRCOIN mints use the same number of decimals. If FUNLESS uses 6 decimals and SRCOIN uses 9 decimals, 1,000,000 funless_amount (1 FUNLESS) yields `1_000_000 * 3 / 10 = 300_000` SRCOIN raw – which is 0.0003 SRC, not 0.3 SRC as intended. The rate is off by 1,000x.

Impact

If the two mints have different decimal configurations, users receive dramatically more or less SRC than the intended 0.3 rate. This can drain the pool or shortchange users by orders of magnitude.

Recommended Fix

Normalize for decimal differences:

```
let decimal_diff = srcoin_mint.decimals as i32 - funless_mint.decimals as i32;
let scale = 10u64.pow(decimal_diff.unsigned_abs());
let srcoin_amount = if decimal_diff >= 0 {
    funless_amount * SWAP_RATE_NUMERATOR * scale / SWAP_RATE_DENOMINATOR
} else {
    funless_amount * SWAP_RATE_NUMERATOR / (SWAP_RATE_DENOMINATOR * scale)
};
```

S-09 – Silent Fee Deduction on Claims

Severity: Medium

Confidence: High

File: `programs/swap_pool/src/lib.rs`

Lines: 259-306

Chain: Solana (Anchor)

Vector Category: Token-2022 Transfer Fee Shortfall

Description

When a user calls `claim`, the function transfers `claimable` SRC from the vault via `transfer_checked`. The SRC mint's 1.29% TransferFeeConfig withholds a portion, so the user receives `claimable * 0.9871`. The booking records `booking.srcoin_claimed = booking.srcoin_entitled` (the full amount), but the user's actual received balance is ~1.29% less. This fee is never disclosed to the user during booking.

Impact

Users receive approximately 1.29% less SRC than their booking entitles them to. For a booking of 150,000 SRC, the shortfall is ~1,935 SRC. Over 500M FUNLESS migrated to 150M SRC, the aggregate shortfall is ~1.935M SRC.

Recommended Fix

Either pre-calculate the net amount at booking time and set `srcoin_entitled` to the post-fee value, or disclose the transfer fee to users with a `msg!` at claim time showing gross vs net. Alternatively, send `claimable / 0.9871` (inverse) so the user receives exactly `claimable` after fee — but this requires the vault to hold extra SRC.

S-10 – CEI Violations – State Updated After CPIs

Severity: Low

Confidence: High

File: `programs/swap_pool/src/lib.rs`

Lines: 57-82, 184-256, 259-306

Chain: Solana (Anchor)

Vector Category: Checks-Effects-Interactions Pattern Violation

Description

In `add_liquidity` (lines 78–79), `book_swap` (lines 239–247), and `claim` (lines 296–302), pool state fields(`total_srcoin_deposited` , `total_funless_booked` , `total_srcoin_booked` , `total_srcoin_claimed`) are updated AFTER the CPI transfer calls. While Solana's runtime prevents direct reentrancy within the same program, the Token-2022 transfer hook fires as a CPI during the transfer, which could observe stale state if it cross-references `swap_pool` accounts.

Impact

In the current codebase, the transfer hook only logs and returns — no cross-program state reads occur. The risk is latent: if the transfer hook or any future CPI reads `swap_pool` state during the transfer, it would see pre-update values. The severity is Low because the current hook is benign.

Recommended Fix

Move all state updates before CPI calls. For `book_swap` , update `booking` and `pool` counters before the `transfer_checked` CPI. Use a pattern where effects are applied first and rolled back on CPI failure.

S-11 – lock_end_ts Overflow is DoS via Panic

Severity: Informational

Confidence: High

File: `programs/swap_pool/src/lib.rs`

Lines: 43

Chain: Solana (Anchor)

Vector Category: Integer Overflow

Description

Line 43 computes `pool.lock_end_ts = pool_start_ts + LOCK_PERIOD_SECONDS` using plain `+` addition. If `pool_start_ts` is set to a value close to `i64::MAX` , the addition panics in release mode (Cargo.toml has `overflow-checks = true`). This does not silently wrap — it causes a transaction failure. The attacker can only grief themselves if they are the initializer.

Impact

No security impact beyond a failed initialization transaction. The panic prevents an invalid lock end timestamp from being stored.

Recommended Fix

Use `checked_add` for consistency and clearer error reporting:

```
pool.lock_end_ts = pool_start_ts.checked_add(LOCK_PERIOD_SECONDS)
    .ok_or(SwapPoolError::ArithmeticOverflow)?;
```

S-12 – booked_at Timestamp Overwritten on Repeat Bookings

Severity: Informational

Confidence: High

File: `programs/swap_pool/src/lib.rs`

Lines: 236

Chain: Solana(Anchor)

Vector Category: Data Overwrite / init_if_needed

Description

The `booking` account uses `init_if_needed`, allowing a user to call `book_swap` multiple times. Line 236 sets `booking.booked_at = clock.unix_timestamp` unconditionally, overwriting the original booking timestamp. While `funless_deposited` and `srcoin_entitled` accumulate correctly via `checked_add`, the `booked_at` field only reflects the last booking time.

Impact

Off-chain systems that rely on `booked_at` to determine booking order or original booking time will see incorrect data. No on-chain logic depends on this field, so the impact is limited to off-chain indexing.

Recommended Fix

Only set `booked_at` on the first booking:

```
if booking.booked_at == 0 {
    booking.booked_at = clock.unix_timestamp;
}
```

S-13 – Claims Allowed During Pause

Severity: Informational

Confidence: High

File: `programs/swap_pool/src/lib.rs`

Lines: 259-306

Chain: Solana(Anchor)

Vector Category: Inconsistent Emergency Stop

Description

The `claim` instruction does not check `pool.is_active`. While `book_swap` is paused when `is_active = false`, claims continue to process. This may be intentional (users shouldn't lose access to locked funds due to admin pause), but it creates inconsistent emergency stop semantics.

Impact

During an emergency (e.g., discovered exploit), the admin cannot prevent claim execution even if the pool is paused. This could be by design to protect user funds.

Recommended Fix

Document the intentional design or add `require!(pool.is_active, SwapPoolError::PoolNotActive)` if claims should also be paused during emergencies.

S-14 – `is_active` Guard Missing on Admin Operations

Severity: Informational

Confidence: Medium

File: `programs/swap_pool/src/lib.rs`

Lines: 57, 97

Chain: Solana (Anchor)

Vector Category: Inconsistent Guard Application

Description

Admin functions `add_liquidity` and `remove_liquidity` do not check `pool.is_active`. An admin can add or remove liquidity even when the pool is paused.

Impact

Minimal – admin operations during pause may be intentional for maintenance. However, it means `is_active` only gates user `book_swap`, creating an inconsistent mental model.

Recommended Fix

Add `require!(pool.is_active, ...)` to admin functions if consistency is desired, or document that `is_active` only gates user bookings.

S-15 – Centralization Risk – Single Admin Key

Severity: Informational

Confidence: High

File: `programs/swap_pool/src/lib.rs`

Lines: 85-93

Chain: Solana (Anchor)

Vector Category: Centralization

Description

The `set_pool_active` function requires only a single admin signature from the `admins` list (line 88: `pool.admins.contains(&ctx.accounts.admin.key())`), without requiring multisig consensus. Any single admin can unilaterally pause or unpause the entire pool.

Impact

A single compromised admin key can permanently pause the pool, preventing all new bookings.

Recommended Fix

Require multisig consensus for `set_pool_active` using the same `remaining_accounts` pattern as `remove_liquidity`, or accept this as an intentional emergency-pause mechanism and document it.

T-01 – Permissionless initialize_crank_config – First Caller Hijacks Fee Governance

Severity: High

Confidence: High

File: `programs/transfer_hook/src/lib.rs`

Lines: 46-59, 356-371

Chain: Solana (Anchor)

Vector Category: Permissionless Initialization

Description

The `InitializeCrankConfig` struct (lines 356-371) uses `init` with `seeds = [b"crank-config"]` and has no access control – any signer can be the `admin`. The first caller permanently becomes the admin who controls the authorized crank wallet. The admin can set themselves as the `authorized_crank`, call `collect_fees`, and retain 60% of all collected fees (the "DAO share") in their own wallet instead of forwarding to the treasury.

Impact

An attacker who front-runs `initialize_crank_config` permanently captures fee governance. All future fee collection goes through their controlled crank wallet. The 60% DAO share is never forwarded to the real treasury.

Recommended Fix

Add an authority check requiring the caller to be the mint authority or a known deployment key:

```
#[account(constraint = admin.key() == EXPECTED_DEPLOYER @ ErrorCode::Unautl
pub admin: Signer<'info>,
```

T-02 – Permissionless initialize_extra_account_meta_list – Unconstrained Mint

Severity: High

Confidence: High

File: `programs/transfer_hook/src/lib.rs`

Lines: 79–136, 394–416

Chain: Solana (Anchor)

Vector Category: Permissionless Init + PDA Seed Parameter

Description

The `InitializeExtraAccountMetaList` struct accepts any `mint: InterfaceAccount<'info, Mint>` without constraining it to be the real SRC mint. The `ExtraAccountMetaList` PDA is derived from `seeds = [b"extra-account-metas", mint.key().as_ref()]`. An attacker can call this with a fake mint, creating a valid `ExtraAccountMetaList` for a different mint, which doesn't affect the real SRC mint. However, the real issue is that the real SRC mint's `ExtraAccountMetaList` can also be initialized by anyone with any `dao_treasury` address, permanently baking a wrong treasury into the hook.

Impact

The first caller to initialize the `ExtraAccountMetaList` for the real SRC mint permanently sets the `dao_treasury` reference. If the attacker provides their own token account as `dao_treasury`, the transfer hook's extra account resolution will reference the attacker's account.

Recommended Fix

Constrain `mint` to a known SRC mint pubkey or require the payer to be the mint authority.

T-03 – amount * 129 Arithmetic Overflow – Transfer DoS Above ~143M Tokens

Severity: Medium

Confidence: High

File: `programs/transfer_hook/src/lib.rs`

Lines: 152-153

Chain: Solana (Anchor)

Vector Category: Integer Overflow

Description

Line 152 computes `fee_estimate = amount * 129 / 10000` using plain `*` multiplication. With `overflow-checks = true` in Cargo.toml, this panics when `amount > u64::MAX / 129 ≈ 1.43 × 1017`. For a token with 9 decimals, this is ~143 million tokens. Any transfer above this amount causes a runtime panic, killing the transaction.

Impact

Transfers exceeding ~143M SRC tokens (14.3% of total supply) will permanently fail. This prevents large DAO treasury movements, large liquidity operations, and any single transfer exceeding this threshold.

Recommended Fix

Use `checked_mul` or cast to `u128`:

```
let fee_estimate = (amount as u128 * 129 / 10000) as u64;
```

T-04 – Pubkey::default() Accepted as authorized_crank

Severity: Medium

Confidence: High

File: `programs/transfer_hook/src/lib.rs`

Lines: 46-59, 62-75

Chain: Solana (Anchor)

Vector Category: Degenerate Value Validation

Description

Both `initialize_crank_config` and `update_crank_config` accept any `Pubkey` as `authorized_crank` without rejecting `Pubkey::default()` (all zeros). If set to

`Pubkey::default()` , the `collect_fees` constraint `crank.key() == crank_config.authorized_crank` can never be satisfied because `Pubkey::default()` cannot sign a transaction. Fee collection is permanently bricked.

Impact

Setting `authorized_crank` to `Pubkey::default()` — whether by accident or malice — permanently prevents fee collection. All withheld fees accumulate in the mint with no way to withdraw, burn, or distribute them.

Recommended Fix

```
require!(authorized_crank != Pubkey::default(), ErrorCode::InvalidCrankAdd)
```

T-05 – No On-Chain DAO Revenue Enforcement

Severity: Medium

Confidence: High

File: `programs/transfer_hook/src/lib.rs`

Lines: 275, 306–311

Chain: Solana (Anchor)

Vector Category: Off-Chain Trust Assumption

Description

After `collect_fees` burns 40% of withheld fees at line 278, the remaining 60% (`dao_share`) stays in `crank_token_account` (line 275). The code comments say "The crank script must send it to `dao_treasury` in the same transaction" (line 198), but this is an off-chain trust assumption. The program has no mechanism to enforce or verify that the crank ever forwards the DAO share.

Impact

The authorized crank can permanently retain the 60% DAO share without forwarding to the treasury. Over time, this accumulates to a significant fraction of all collected fees.

Recommended Fix

Instead of leaving DAO share in the crank's account, transfer it directly to `dao_treasury` within the `collect_fees` instruction using a second CPI transfer. This makes the fee split atomic and trustless.

T-06 – Arithmetic Overflow in collect_fees Fee Split

Severity: Medium

Confidence: High

File: `programs/transfer_hook/src/lib.rs`

Lines: 247, 262, 269, 299–300

Chain: Solana (Anchor)

Vector Category: Integer Overflow

Description

Line 247 computes `burn_share = withheld * 40 / 100` using `plain *`. With `overflow-checks = true`, this panics if `withheld > u64::MAX / 40  $\approx 4.6 \times 10^{17}$` . Lines 262 and 269 also use `plain +` for `burned_this_year + burn_share` and `total_burned + burn_share`. Lines 299–300 use `.unwrap()` on `checked_add`, which also panics on overflow.

Impact

If accumulated withheld fees exceed the overflow threshold, `collect_fees` permanently fails. Fees accumulate in the mint with no way to collect, burn, or distribute them.

Recommended Fix

Use `u128` intermediaries for the split calculation and replace `.unwrap()` with `.ok_or()`:

```
let burn_share = ((withheld as u128) * 40 / 100) as u64;
limiter.burned_this_year = limiter.burned_this_year.checked_add(burn_share)
    .ok_or(ErrorCode::ArithmeticOverflow)?;
```

T-07 – Unconstrained mint in harvest_fees and collect_fees

Severity: Medium

Confidence: High

File: `programs/transfer_hook/src/lib.rs`

Lines: 466–478, 484–535

Chain: Solana (Anchor)

Vector Category: Unconstrained Account

Description

Both `HarvestFees` and `CollectFees` accept any `mint: InterfaceAccount<'info, Mint>` without constraining it to the real SRC mint. An attacker can create a worthless token with a `TransferFeeConfig`, accumulate withheld fees on it, and use `harvest_fees` / `collect_fees` to process fees for the wrong mint. The `BurnLimiter` is a global singleton — processing a fake mint's fees updates the real burn counters.

Impact

An attacker can manipulate the `BurnLimiter` by processing fake mint fees, prematurely exhausting the burn cap. Once `total_burned` reaches `MAX_TOTAL_BURN`, no more legitimate SRC fees can be burned, shifting the protocol permanently to 100% treasury mode earlier than intended.

Recommended Fix

Add a known SRC mint constant and validate:

```
pub const SRC_MINT: Pubkey = pubkey!("...");

#[account(mut, address = SRC_MINT @ ErrorCode::InvalidMint)]
pub mint: InterfaceAccount<'info, Mint>,
```

T-08 — dao_treasury Permanently Baked into ExtraAccountMetaList

Severity: Low

Confidence: High

File: `programs/transfer_hook/src/lib.rs`

Lines: 88–99

Chain: Solana (Anchor)

Vector Category: Immutable Configuration

Description

The `dao_treasury` pubkey is stored in the `ExtraAccountMetaList` at index 5 as a fixed pubkey via `ExtraAccountMeta::new_with_pubkey` (line 90). There is no update instruction. If the DAO treasury needs to change (key rotation, compromise, governance decision), the `ExtraAccountMetaList` cannot be updated without redeploying the entire program and reinitializing the PDA.

Impact

The DAO treasury address referenced by the transfer hook is permanently immutable. Any future treasury migration requires a full program upgrade.

Recommended Fix

Use a PDA-based indirection: store the treasury address in the `CrankConfig` account (which has `update_crank_config`), and use `ExtraAccountMeta::new_with_seeds` to resolve it dynamically. Alternatively, add an `update_extra_account_meta_list` instruction.

T-09 – Silent u64 Truncation – as u64 Narrowing Casts

Severity: Low

Confidence: High

File: `programs/transfer_hook/src/lib.rs`

Lines: 251

Chain: Solana (Anchor)

Vector Category: Unsafe Type Cast

Description

Line 251 computes `current_year = ((clock.unix_timestamp - GENESIS_TIMESTAMP) / SECONDS_PER_YEAR) as u64`. The `as u64` cast silently truncates if the i64 intermediate is negative (which can happen if `unix_timestamp < GENESIS_TIMESTAMP`). Before `GENESIS_TIMESTAMP` (Jan 1, 2024), the year would wrap to a very large u64 value.

Impact

If the program were deployed before Jan 1, 2024, the year counter would be incorrect. Since the program is deployed in 2026, this is not currently exploitable. The severity is Low as a defensive coding concern.

Recommended Fix

```
let elapsed = clock.unix_timestamp.checked_sub(GENESIS_TIMESTAMP)
    .ok_or(ErrorCode::InvalidTimestamp)?;
let current_year = (elapsed / SECONDS_PER_YEAR) as u64;
```

T-10 – DAO Fee Share Left in Crank Wallet

Severity: Low

Confidence: High

File: `programs/transfer_hook/src/lib.rs`

Lines: 275, 306–311

Chain: Solana (Anchor)

Vector Category: Incomplete Fee Distribution

Description

After burning the burn portion, `dao_share = withheld - burn_share` remains in the crank's token account (line 275). The crank wallet is trusted to forward this to the DAO treasury off-chain. There is no on-chain receipt, no deadline, and no penalty for non-forwarding.

Impact

The DAO share of fees depends entirely on the crank operator's honesty and operational reliability. If the crank wallet is compromised or abandoned, DAO share accumulates in an inaccessible wallet.

Recommended Fix

Transfer `dao_share` directly to `dao_treasury` within `collect_fees` using a CPI, making the split atomic.

T-11 – Permissionless initialize_burn_limiter

Severity: Informational

Confidence: High

File: `programs/transfer_hook/src/lib.rs`

Lines: 34–41, 339–354

Chain: Solana (Anchor)

Vector Category: Permissionless Initialization

Description

`InitializeBurnLimiter` has no access control – any signer can initialize it. However, the BurnLimiter PDA is deterministic (`seeds = [b"burn-limiter"]`) and the initialization sets all counters to 0. A front-runner gains nothing – they simply pay rent for an account that must exist. The values cannot be manipulated.

Impact

No security impact. The front-runner pays rent and gets nothing. The correct initial state (all zeros) is enforced by the code.

Recommended Fix

None required. Optionally add access control for defense-in-depth.

T-13 – fallback Lacks Token-2022 Caller Validation

Severity: Informational

Confidence: Medium

File: `programs/transfer_hook/src/lib.rs`

Lines: 318–336

Chain: Solana (Anchor)

Vector Category: Open Entry Point

Description

The `fallback` function dispatches to `__private::__global::transfer_hook` (line 332), which is Anchor's internal dispatch. This works correctly for Token-2022 CPI calls, but the fallback does not independently validate that the caller is the Token-2022 program. It relies on Anchor's private API (`__private::__global`), which could change between Anchor versions.

Impact

Currently functional. The risk is forward-compatibility: an Anchor version upgrade could break the dispatch path silently.

Recommended Fix

Add explicit caller validation:

```
let token_program = accounts.first().ok_or(ProgramError::NotEnoughAccountKeys)
require!(token_program.key == &spl_token_2022::id(), ProgramError::IncorrectProgramId)
```

T-14 – associated_token_program Unused

Severity: Informational

Confidence: High

File: `programs/transfer_hook/src/lib.rs`

Lines: 414

Chain: Solana (Anchor)

Vector Category: Dead Code

Description

The `InitializeExtraAccountMetaList` struct includes `associated_token_program: Program<'info, AssociatedToken>` at line 414, but no instruction in the function creates an ATA or uses this program.

Impact

Wasted transaction space — the client must pass an extra account that is never used.

Recommended Fix

Remove the `associated_token_program` field from the struct.

T-15 – Fee Estimates Are Approximations

Severity: Informational

Confidence: High

File: `programs/transfer_hook/src/lib.rs`

Lines: 152–159

Chain: Solana (Anchor)

Vector Category: Misleading Information

Description

The `msg!` output at lines 155–159 logs `est_fee` and `est_burn` computed via `amount * 129 / 10000`. The actual withheld fee is computed by the Token-2022 TransferFeeConfig extension using its own formula (which may differ due to rounding, ceiling division, and the exact basis points configured). The logged values are estimates, not actual withheld amounts.

Impact

Off-chain indexers relying on these log values will see slightly inaccurate fee data. No on-chain impact.

Recommended Fix

Rename log fields to `est_fee_approx` and `est_burn_approx`, or compute actual values from the mint's TransferFeeConfig extension.

T-16 – Silent Mode Change When Burn Cap Is Hit

Severity: Informational

Confidence: High

File: `programs/transfer_hook/src/lib.rs`

Lines: 268-273

Chain: Solana (Anchor)

Vector Category: Undisclosed Economic Shift

Description

When `limiter.total_burned + burn_share > MAX_TOTAL_BURN` (line 269), `burn_share` is reduced to `remaining`, which may be 0. At that point, `dao_share = withheld - 0 = withheld`, meaning 100% of fees go to the crank (intended for DAO). No event or log signals that the protocol has permanently exited its deflationary phase.

Impact

The token's economic model silently shifts from 40/60 burn/DAO to 0/100. Holders expecting ongoing deflation are not notified.

Recommended Fix

Emit a distinct event when `burn_share` is capped to 0:

```
if remaining == 0 {  
    msg!("BURN CAP REACHED: All future fees go 100% to DAO");  
}
```

T-17 – No Reserved Space in State Structs

Severity: Informational

Confidence: High

File: `programs/transfer_hook/src/lib.rs`

Lines: 537-561

Chain: Solana (Anchor)

Vector Category: Upgrade Compatibility

Description

`BurnLimiter` (24 bytes) and `CrankConfig` (64 bytes) have no reserved bytes. Adding new fields in a future upgrade requires `realloc`, which needs rent-payer coordination and a new instruction.

Impact

Future state schema changes are more complex than necessary.

Recommended Fix

Add reserved bytes:

```
pub struct CrankConfig {  
    pub authorized_crank: Pubkey,  
    pub admin: Pubkey,  
    pub _reserved: [u8; 64],  
}
```

P-01 – Protocol Paralysis – Missing Transfer Hook Extra Accounts

Severity: High

Confidence: High

File: `programs/sol_pool/src/lib.rs`

Lines: 70-82, 149-161, 242-255, 382-395

Chain: Solana (Anchor)

Vector Category: Token-2022 Transfer Hook Incompatibility

Description

All SRC token transfers in `sol_pool` use `token::transfer_checked` (Anchor CPI) without providing the extra accounts required by the Token-2022 TransferHook extension. The SRC mint requires `ExtraAccountMetaList`, `dao_treasury`, and `burn_limiter` accounts. Without them, every outgoing SRC transfer fails.

Impact

`initialize_pool` (SRC deposit), `add_src_liquidity`, `remove_src_liquidity`, `swap_sol_for_src` (SRC withdrawal), and `swap_src_for_sol` (SRC deposit) all fail. The AMM pool is completely non-functional for SRC operations.

Recommended Fix

Same as S-02: replace `token::transfer_checked` with manual instruction building + `add_extra_accounts_for_execute_cpi`.

P-02 – Unconstrained src_coin_mint – Fake Mint Attack

Severity: High

Confidence: High

File: `programs/sol_pool/src/lib.rs`

Lines: 595, 619, 667, 729, 764

Chain: Solana (Anchor)

Vector Category: Unconstrained Mint

Description

The `srcoin_mint` account in `InitializePool`, `AddSrcLiquidity`, `RemoveSrcLiquidity`, `SwapSolForSrc`, and `SwapSrcForSol` is declared as `pub srcoin_mint: InterfaceAccount<'info, Mint>` with no `address` or `has_one` constraint binding it to `pool_config.srcoin_mint`. An attacker can pass a fake mint with matching decimals, deposit worthless tokens, and withdraw real SOL.

Impact

In `swap_src_for_sol`: An attacker creates a fake SRC mint, deposits fake tokens into a substituted vault, and withdraws real SOL from the pool. The entire SOL reserve can be drained.

Recommended Fix

Add `has_one = srcoin_mint` to `pool_config` in every instruction struct:

```
#[account(mut, seeds = [b"sol-pool-config"], bump, has_one = srcoin_mint)]
pub pool_config: Account<'info, SolPoolConfig>,
```

P-03 – Vault Substitution – Missing `has_one`

Severity: High

Confidence: High

File: `programs/sol_pool/src/lib.rs`

Lines: 598-603, 621-626, 669-674, 731-736, 766-771

Chain: Solana (Anchor)

Vector Category: Account Substitution

Description

The `srcoin_vault` and `sol_vault` accounts are validated via `token::mint` and `token::authority` constraints but are not bound to `pool_config.srcoin_vault` or `pool_config.sol_vault` via `has_one`. An attacker can pass any token account that matches the mint and authority, redirecting swap outputs.

Impact

An attacker can substitute the SRC vault during `swap_sol_for_src`, causing SRC to be sent from a different source while the pool's state tracks against the real vault. The SOL vault is a PDA with deterministic seeds(`[b"sol-vault"]`), making substitution harder but still possible if the check relies only on `seeds = [b"sol-vault"]` without `has_one`.

Recommended Fix

Add `has_one = srcoin_vault` and `has_one = sol_vault` to all relevant instruction structs.

P-04 – Slippage Protection Ineffective After Transfer Fee

Severity: Medium

Confidence: High

File: `programs/sol_pool/src/lib.rs`

Lines: 363, 471

Chain: Solana (Anchor)

Vector Category: Slippage Check Ordering

Description

In `swap_sol_for_src`, the slippage check `require!(src_out >= min_src_out)` at line 363 validates the gross output. The subsequent `transfer_checked` CPI at line 382 withholds ~1.29% via `TransferFeeConfig`. The user receives `src_out * 0.9871`, which may be below their `min_src_out`. In `swap_src_for_sol`, the user correctly receives net SOL (no fee on SOL), but the slippage check also passes on gross `sol_out` before the actual transfer.

Impact

Users who set tight slippage tolerance receive less SRC than `min_src_out` by approximately 1.29%. A user swapping 100 SOL expecting 1,000 SRC (`min_src_out = 990`) could receive ~987 SRC, below their threshold, yet the transaction succeeds.

Recommended Fix

Adjust the slippage check to account for the transfer fee:

```
let net_src_out = src_out * (10000 - 129) / 10000;
require!(net_src_out >= min_src_out, SolPoolError::SlippageExceeded);
```

P-05 – CEI Violations in Swap and Liquidity Functions

Severity: Medium

Confidence: High

File: `programs/sol_pool/src/lib.rs`

Lines: 397-408, 491-502

Chain: Solana (Anchor)

Vector Category: Checks-Effects-Interactions Pattern Violation

Description

In `swap_sol_for_src` (lines 397-408) and `swap_src_for_sol` (lines 491-502), pool reserves, swap counters, and volume trackers are updated AFTER the CPI transfers. The Token-2022 transfer hook fires during the CPI, which could observe stale reserve values.

Impact

Similar to S-10 – currently benign since the transfer hook only logs. The risk is latent.

Recommended Fix

Move reserve updates before CPI calls.

P-06 – Transfer Fee Shortfall in Remove Liquidity

Severity: Medium

Confidence: High

File: `programs/sol_pool/src/lib.rs`

Lines: 242-258

Chain: Solana (Anchor)

Vector Category: Token-2022 Transfer Fee Shortfall

Description

`remove_src_liquidity` updates `pool.src_reserve = new_reserve` (line 258) where `new_reserve = pool.src_reserve - amount_lamports` (line 230). However, the actual amount transferred out is `amount_lamports` minus the 1.29% transfer fee – the vault retains ~1.29% as withheld fees. The reserve counter decrements by the full gross amount, creating a permanent deficit between tracked reserves and actual vault balance.

Impact

Over multiple remove operations, `pool.src_reserve` increasingly underestimates actual vault balance. The AMM constant-product formula uses stale reserves, producing incorrect swap outputs.

Recommended Fix

Reload the vault after transfer and compute actual change:

```
ctx.accounts.srcoin_vault.reload()?;  
let actual_sent = amount_lamports - (ctx.accounts.srcoin_vault.amount - exp  
pool.src_reserve = pool.src_reserve.checked_sub(actual_sent).unwrap();
```

P-07 – Silent u64 Truncation in SRC Lamport Scaling

Severity: Low

Confidence: High

File: `programs/sol_pool/src/lib.rs`

Lines: 58-60, 143-145

Chain: Solana (Anchor)

Vector Category: Unsafe Type Cast

Description

Lines 58-60 compute `src_lamports = (src_amount as u128).checked_mul(10u128.pow(decimals as u32))... as u64`. The final `as u64` silently truncates if the u128 result exceeds `u64::MAX`. For 9 decimals, this happens when `src_amount > 18,446,744,073` (~18.4 billion tokens), which exceeds the 1B total supply.

Impact

Not exploitable with the current 1B supply. The truncation would only trigger with amounts far exceeding the protocol's tokenomics.

Recommended Fix

Add a bounds check before the cast:

```
require!(src_lamports <= u64::MAX as u128, SolPoolError::ArithmeticOverflow)
```

P-08 – Lock Floor Incompatibility With Swap Depletion

Severity: Low

Confidence: Medium

File: `programs/sol_pool/src/lib.rs`

Lines: 229-237

Chain: Solana (Anchor)

Vector Category: Economic Design

Description

The 82-year lock check (lines 232-236) requires `new_reserve >= pool.initial_src_deposit` before admin can remove SRC. However, normal swap activity can deplete `src_reserve` below `initial_src_deposit` via `swap_sol_for_src` (users withdraw SRC). When the reserve is swap-depleted below the initial deposit, the admin cannot remove ANY SRC even after the lock expires, because the check uses `src_reserve` (post-swap) rather than tracking admin additions separately.

Impact

If heavy swap activity pushes SRC reserves below the initial deposit, admin liquidity removal is blocked until sufficient SRC flows back via reverse swaps. This is a usability issue, not an exploit.

Recommended Fix

Track admin-deposited liquidity separately from swap-driven reserve changes:

```
pub admin_src_added: u64, // total admin deposits beyond initial
```

And check against `initial_src_deposit - admin_src_added` or similar accounting.

P-09 – Rounding Inconsistency Between `quote_swap` and Execution

Severity: Low

Confidence: High

File: `programs/sol_pool/src/lib.rs`

Lines: 516-540, 347-352

Chain: Solana (Anchor)

Vector Category: Rounding Mismatch

Description

The `quote_swap` function at line 526 computes fees using floor division: `fee = input_amount * 30 / 10000`. The execution path in `swap_sol_for_src` at lines 347-352 uses ceiling division: `fee`

`= (input_amount * 30 + 9999) / 10000` . For the same input, the quote returns a slightly higher output than actual execution produces.

Impact

Users who set `min_src_out` based on the quote output may have their swaps fail because actual output is 1 lamport less. This primarily affects very small swaps where the rounding difference is proportionally significant.

Recommended Fix

Use the same ceiling division in `quote_swap` :

```
let fee = input_amount.checked_mul(SWAP_FEE_BPS)?.checked_add(BPS_DENOMINATOR)
```

P-10 – Misleading Error Code and Dead Guard in `compute_output`

Severity: Low

Confidence: High

File: `programs/sol_pool/src/lib.rs`

Lines: 549-564

Chain: Solana (Anchor)

Vector Category: Code Quality

Description

`compute_output` uses `SolPoolError::ArithmeticOverflow` for all error cases, including division by zero (when `input_reserve + input_amount = 0` , which requires both to be 0 – impossible after the `input_amount > 0` check in callers). The function is correct but the error naming is misleading for a zero-reserve edge case that can never occur.

Impact

No functional impact. Misleading error code could confuse debugging.

Recommended Fix

The function is safe as-is. Optionally rename the error or add distinct error variants.

P-11 – Volume Counter Bricks Protocol at `u64::MAX`

Severity: Low

Confidence: High

File: `programs/sol_pool/src/lib.rs`

Lines: 403–408, 497–502

Chain: Solana (Anchor)

Vector Category: Counter Overflow / DoS

Description

`total_swaps`, `total_sol_volume`, and `total_src_volume` use `checked_add` (lines 403–408, 497–502), which returns `None` and triggers `ArithmeticOverflow` when reaching `u64::MAX`.

These are purely observational counters with no impact on pool logic, yet their overflow permanently prevents all swaps.

Impact

At `u64::MAX` for any counter, all subsequent swaps fail. For `total_swaps`, this requires 1.8×10^{19} swaps — practically unreachable. For `total_src_volume` with 9 decimals, it overflows after ~18.4 billion SRC cumulative volume — also unlikely but theoretically possible over 82 years.

Recommended Fix

Use `saturating_add` for observational counters:

```
pool.total_swaps = pool.total_swaps.saturating_add(1);
```

P-12 – CEI Violation and Misleading Log in `initialize_pool`

Severity: Informational

Confidence: High

File: `programs/sol_pool/src/lib.rs`

Lines: 125–131

Chain: Solana (Anchor)

Vector Category: Log Accuracy

Description

The `msg!` at line 126 logs `src_amount` (the whole-token input) rather than

`src_actually_received` (the post-fee lamport value that was actually deposited). The state correctly stores `src_actually_received`, but the log is misleading.

Impact

Off-chain indexers see the gross amount, not the net amount.

Recommended Fix

Log `src_actually_received` instead.

P-13 – Access Control Asymmetry

Severity: Informational

Confidence: High

File: `programs/sol_pool/src/lib.rs`

Lines: 134–175, 210–262

Chain: Solana (Anchor)

Vector Category: Governance Inconsistency

Description

`add_src_liquidity` and `add_sol_liquidity` require only single admin signature.
`remove_src_liquidity` and `remove_sol_liquidity` require multisig consensus. This asymmetry means a single compromised admin can inflate reserves (by depositing) but cannot withdraw.

Impact

Low – inflation requires the admin to actually possess tokens to deposit. The asymmetry is likely intentional (easy in, hard out).

Recommended Fix

Document the intentional design or add multisig to add functions for consistency.

P-14 – Unilateral Admin Pause

Severity: Informational

Confidence: High

File: `programs/sol_pool/src/lib.rs`

Lines: 320–329

Chain: Solana (Anchor)

Vector Category: Centralization

Description

`set_pool_active` requires only single admin signature, not multisig. Any single admin can freeze all swap operations.

Impact

A compromised admin key can indefinitely freeze the pool.

Recommended Fix

Either accept as an intentional emergency mechanism or require multisig for state-changing admin actions.

P-15 – DEFAULT Constants Never Enforced

Severity: Informational

Confidence: High

File: `programs/sol_pool/src/lib.rs`

Lines: 13-16, 45-46

Chain: Solana (Anchor)

Vector Category: Unused Constants

Description

`DEFAULT_SRC_AMOUNT` (7.5M) and `DEFAULT_SOL_AMOUNT` (500) are declared but never referenced in any `require!` or validation check. The pool can be initialized with `src_amount = 1` and `sol_amount = 1`.

Impact

A pool initialized with dust amounts has extreme slippage on every swap. The constants suggest minimum intended values that are not enforced.

Recommended Fix

Add minimum validation:

```
require!(src_amount >= DEFAULT_SRC_AMOUNT, SolPoolError::AmountTooSmall);
require!(sol_amount >= DEFAULT_SOL_AMOUNT, SolPoolError::AmountTooSmall);
```

P-16 – Dead Liquidity – Pre-Existing Vault Balance Ignored

Severity: Informational

Confidence: High

File: `programs/sol_pool/src/lib.rs`

Lines: 68, 86–88, 113–116

Chain: Solana (Anchor)

Vector Category: Accounting Gap

Description

If the SRC vault already holds tokens before `initialize_pool` is called, the pre-existing balance is not included in `initial_src_deposit` or `src_reserve`. Only the amount transferred during initialization (`src_actually_received`) is tracked. The pre-existing tokens become "dead liquidity" — present in the vault but invisible to the AMM math.

Impact

Dead liquidity creates a discrepancy between vault balance and tracked reserves. Swap outputs are computed on tracked reserves, so the dead tokens are never traded.

Recommended Fix

Either require the vault to be empty at initialization or include the full vault balance:

```
pool.src_reserve = ctx.accounts.srcoin_vault.amount; // full balance
```

P-17 – No Structured Event Emission

Severity: Informational

Confidence: High

File: `programs/sol_pool/src/lib.rs`

Lines: All functions

Chain: Solana (Anchor)

Vector Category: Observability

Description

All functions use `msg!` for logging, which produces unstructured text in transaction logs. Anchor `# [event]` structs are not used anywhere. This makes off-chain indexing fragile and dependent on parsing text formats.

Impact

No security impact. Reduced operational visibility and indexer reliability.

Recommended Fix

Define Anchor `#[event]` structs for Swap, AddLiquidity, RemoveLiquidity, etc., and emit them instead of `msg!`.

R-01 – No Total Supply Cap Validation

Severity: High

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 35–86

Chain: Solana (Anchor)

Vector Category: Missing Invariant Enforcement

Description

`initialize_token_with_hook` accepts `owner_amount`, `dao_amount`, `unlock_amount` as unconstrained u64 parameters (lines 37–39). These are minted at lines 50–68, then mint authority is revoked at lines 71–85. The whitepaper states total supply is 1,000,000,000 SRCOIN, but no validation enforces this sum. The first caller can mint any total.

Impact

The deployed token could have an arbitrary supply – 999 billion, 1 token, or any value – permanently locked after mint authority revocation.

Recommended Fix

```
const TOTAL_SUPPLY: u64 = 1_000_000_000 * 1_000_000_000; // 1B with 9 decir
require!(
    owner_amount.checked_add(dao_amount)
        .and_then(|s| s.checked_add(unlock_amount))
        .ok_or(CustomError::ArithmeticOverflow)? == TOTAL_SUPPLY,
    CustomError::InvalidSupply
);
```

R-02 – Permissionless initialize_token_with_hook

Severity: High

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 35–94, 591–620

Chain: Solana (Anchor)

Vector Category: Permissionless Initialization

Description

`InitializeToken` (lines 591–620) requires only a `Signer` for `authority` with no constraint binding it to a known deployer. The `dao_treasury_multisig` PDA uses `seeds = [b"dao-multisig"]` with `init`, so the first caller permanently captures the DAO multisig, sets the signer list, chooses the threshold, and determines how the entire 1B supply is distributed.

Impact

A front-runner can initialize the token with their own signer set, mint the entire supply to their wallets, and permanently own governance.

Recommended Fix

Constrain `authority` to a known deployer pubkey or use a two-step initialization with a pre-set admin PDA.

R-03 – Permissionless initialize_unlock_manager

Severity: High

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 96–122, 622–633

Chain: Solana (Anchor)

Vector Category: Permissionless Initialization

Description

`InitializeUnlockManager` (lines 622–633) uses `init` with `seeds = [b"unlock-manager"]` and no access control. The first caller permanently sets all vesting plans – categories, amounts, durations, cliffs, and price targets. The `authority` signer only pays rent.

Impact

An attacker can set vesting schedules with immediate unlocks (`start_ts = 0` , `cliff_ts = 0` , `period_duration = 1`), enabling instant withdrawal of the entire vesting allocation.

Recommended Fix

Require the `authority` signer to match a known deployer or be verified against the `DaoTreasuryMultisig`.

R-04 – Caller-Supplied released Field in UnlockPlan

Severity: High

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 96-122, 857

Chain: Solana (Anchor)

Vector Category: State Initialization Corruption

Description

The `UnlockPlan` struct (line 854) has a `released: u64` field. When plans are passed as `unlock_plans: Vec<UnlockPlan>` in `initialize_unlock_manager`, the caller can set `released` to any value. If `released = total_allocated`, the vesting check at line 214 (`available = unlocked_total.saturating_sub(plan.released)`) yields 0 for all future unlocks, permanently locking the funds.

Impact

A malicious initializer can set `released` to block all future unlocks for specific categories, permanently trapping tokens in the vault.

Recommended Fix

Force `released = 0` during initialization:

```
for plan in unlock_manager.plans.iter_mut() {
    plan.released = 0;
}
```

R-05 – start_ts = 0 Not Validated – Complete Vesting Bypass

Severity: High

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 96-122, 199-213

Chain: Solana (Anchor)

Vector Category: Time-Based Bypass

Description

`initialize_unlock_manager` does not validate `start_ts` or `cliff_ts` values. If `start_ts = 0` and `cliff_ts = 0`, the effective start is Unix epoch (January 1, 1970). At line 204, `elapsed = now.saturating_sub(0) = now` (billions of seconds). At line 208, `unlocked_periods = min(elapsed / period_duration, period_count)` immediately equals `period_count`, making the entire allocation available for instant withdrawal.

Impact

The entire vesting schedule can be bypassed by setting `start_ts = 0`, allowing immediate withdrawal of all 842M tokens.

Recommended Fix

```
require!(plan.start_ts > 0, CustomError::InvalidPlanConfig);  
require!(plan.start_ts >= clock.unix_timestamp - 86400, CustomError::InvalidPlanConfig);
```

R-06 – NFT Registry Exhaustion – 100-Entry Cap

Severity: High

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 27, 416-419

Chain: Solana (Anchor)

Vector Category: Resource Exhaustion

Description

`MAX_NFT_REGISTRY_ENTRIES = 100` (line 27) caps the global NFT registry. Combined with the unconstrained `src_mint` (R-13), an attacker can use a fake SRC mint with inflated balance to claim 100 NFTs (50 per tier), filling the registry at near-zero cost. After 100 entries, `require!(registry.entries.len() < 100)` at line 417 permanently blocks all future claims.

Impact

All legitimate governance NFT claims are permanently disabled after 100 attacker claims. Governance participation is impossible.

Recommended Fix

See R-13 (fix fake mint first). Then either increase the cap using paginated registries or use per-tier counters instead of a growing Vec.

R-07 – No Duplicate Signer Validation in initialize_token_with_hook

Severity: Medium

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 35-94

Chain: Solana (Anchor)

Vector Category: Governance Deadlock

Description

`dao_signers: Vec<Pubkey>` is stored directly at line 90 without deduplication. If `[keyA, keyA, keyA]` is passed with `threshold = 3`, the multisig checks deduplicate and count `keyA` as 1 approval, which never meets the threshold of 3. Governance is permanently deadlocked.

Impact

Permanent governance freeze – no DAO transfers, no signer updates, no vesting unlocks can ever pass multisig.

Recommended Fix

```
let mut unique = dao_signers.clone();
unique.sort();
unique.dedup();
require!(unique.len() == dao_signers.len(), CustomError::DuplicateSigner);
```

R-08 – No Duplicate Signer Validation in dao_update_signers

Severity: Medium

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 340–367

Chain: Solana (Anchor)

Vector Category: Governance Deadlock

Description

Same as R-07 but in `dao_update_signers`. Post-deployment, a careless or malicious signer update can introduce duplicates, permanently freezing governance.

Recommended Fix

Same duplicate detection as R-07.

R-09 – Atomic Full Signer Replacement With No Timelock

Severity: Medium

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 340–367

Chain: Solana (Anchor)

Vector Category: Governance Takeover

Description

`dao_update_signers` replaces the entire signer set and threshold in a single atomic operation (lines 363–364). A single call that meets the current threshold can replace all signers with a single attacker key and set `threshold = 1`, permanently capturing governance.

Recommended Fix

Add a timelock: store pending signers with an activation timestamp, require a second confirmation transaction after a delay (e.g., 48 hours).

R-10 – Duplicate Category Names Create Unreachable Plans

Severity: Medium

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 151–219

Chain: Solana (Anchor)

Vector Category: State Corruption

Description

`unlock_funds` iterates plans and matches `plan.category == category` at line 152, breaking on the first match (line 219). If two plans share the same category name, the second is permanently unreachable. Its allocated tokens are locked forever.

Recommended Fix

Validate uniqueness during initialization:

```
let categories: Vec<&str> = unlock_plans.iter().map(|p| p.category.as_str());
let mut sorted = categories.clone();
sorted.sort();
sorted.dedup();
require!(sorted.len() == categories.len(), CustomError::DuplicateCategory),
```

R-11 – Missing Plan Count Limit and Category String Length Validation

Severity: Medium

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 96-122, 626

Chain: Solana (Anchor)

Vector Category: Space Overflow

Description

No limit on `unlock_plans.len()` and no limit on `category` string length. The account space at line 626 allocates for 10 plans × 104 bytes. Passing 11+ plans or a plan with a category string longer than 20 characters would exceed the allocated space, causing the transaction to fail. However, if space is enlarged to accommodate, unbounded plans create unbounded compute in the `for` loop in `unlock_funds`.

Recommended Fix

```
require!(unlock_plans.len() <= 10, CustomError::TooManyPlans);
for plan in &unlock_plans {
    require!(plan.category.len() <= 20, CustomError::CategoryTooLong);
}
```

R-12 – Unconstrained mint in unlock_funds

Severity: Medium

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 636-671

Chain: Solana (Anchor)

Vector Category: Unconstrained Mint

Description

The `UnlockFunds` struct accepts `mint: InterfaceAccount<'info, Mint>` without constraining it to the real SRC mint. An attacker can pass a fake mint, matching `unlock_vault` to a vault holding fake tokens. The `unlock_manager.released` counter increments against the real vesting budget, but the actual SRC in the real vault is untouched. This exhausts the vesting budget without distributing real tokens.

Recommended Fix

Add `address = SRC_MINT` constraint on the mint account, or bind `unlock_vault` to the unlock manager via `has_one`.

R-13 – Unconstrained src_mint in claim_nft

Severity: Medium

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 741

Chain: Solana (Anchor)

Vector Category: Unconstrained Mint / NFT Forgery

Description

The `ClaimNft` struct accepts `src_mint: InterfaceAccount<'info, Mint>` at line 741 without constraining it to the real SRC mint. An attacker can create a fake mint, self-mint 150,000+ tokens, and claim governance NFTs using the fake balance. The balance check at line 394 (`user_balance >= required`) passes because the fake token account holds sufficient fake tokens.

Recommended Fix

Add a known SRC mint constant and validate:

```
pub const SRC_MINT: Pubkey = pubkey!("...");

#[account(address = SRC_MINT)]
pub src_mint: InterfaceAccount<'info, Mint>
```

R-14 – Price Escalation Bypass via Batch Claiming

Severity: Medium

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 169–196

Chain: Solana(Anchor)

Vector Category: Price Gate Bypass

Description

The price escalation check at line 178 computes `next_period = released_periods + 1` and validates `current_price >= required_price` for that specific period. If a caller unlocks multiple periods worth in a single call (by setting `amount` to cover periods 1 through N), only the price for `next_period` (period 1) is checked. Periods 2 through N, which have higher required prices, are not individually validated.

Recommended Fix

Validate the price against the LAST period being unlocked, not the first:

```
let last_period = released_periods + (amount / per_period);
let required_price = compute_escalating_price(..., last_period)?;
```

R-15 – `per_period = 0` Freezes Price Escalation

Severity: Medium

Confidence: Medium

File: `programs/src_token/src/lib.rs`

Lines: 169–176

Chain: Solana(Anchor)

Vector Category: Division Edge Case

Description

`per_period = plan.total_allocated / plan.period_count` (line 170). If `total_allocated < period_count` (e.g., 1 token allocated over 82 periods), `per_period = 0`. Then `released_periods = plan.released / 0`, which would panic – but line 173 guards this with `if per_period > 0`. When `per_period == 0`, `released_periods = 0` and `next_period = 1` permanently, so the price check always uses period 1's base price, bypassing all escalation.

Recommended Fix

Require `per_period > 0`:

```
let per_period = plan.total_allocated.checked_div(plan.period_count)
    .ok_or(CustomError::ArithmeticOverflow)?;
require!(per_period > 0, CustomError::InvalidPlanConfig);
```

R-16 – Sybil Attack on NFT Gating via Flash Loan

Severity: Medium

Confidence: Medium

File: `programs/src_token/src/lib.rs`

Lines: 394–403

Chain: Solana (Anchor)

Vector Category: Flash Loan / Instantaneous Check

Description

The balance check at line 394 is a point-in-time snapshot. An attacker can borrow SRC via a flash loan from a DeFi protocol, claim the NFT, and repay in the same transaction. The whitepaper intends LOCKING, not just holding. Without a lock mechanism, the balance check is bypassable.

Recommended Fix

Implement the whitepaper's lock mechanism (see WP-06) or require the balance to be maintained over a lookback window.

R-17 – `unlock_vault` Has No Canonical Address Binding

Severity: Low

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 639–640

Chain: Solana (Anchor)

Vector Category: Account Substitution

Description

`unlock_vault` in `UnlockFunds` is declared as `InterfaceAccount<'info, TokenAccount>` with no `has_one` or PDA constraint. Any token account can be passed as the vault.

Recommended Fix

Store the unlock vault address in `UnlockManager` and add `has_one = unlock_vault`.

R-18 – dao_treasury Has No Canonical Address Binding

Severity: Low

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 674–706

Chain: Solana (Anchor)

Vector Category: Account Substitution

Description

`dao_treasury` in `DaoTransfer` is declared as `InterfaceAccount<'info, TokenAccount>` without `has_one` binding to the `DaoTreasuryMultisig`. Any token account can be passed as the treasury source.

Recommended Fix

Store the treasury address in `DaoTreasuryMultisig` and add `has_one = dao_treasury`.

R-19 – Self-Transfer in dao_transfer

Severity: Low

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 275–338

Chain: Solana (Anchor)

Vector Category: Economic Leak

Description

`dao_transfer` does not prevent `destination == dao_treasury`. A self-transfer triggers the 1.29% TransferFeeConfig, withholding fees from the treasury's own balance. The treasury slowly leaks value to withheld fees.

Recommended Fix

```
require!(ctx.accounts.destination.key() != ctx.accounts.dao_treasury.key(),
```

R-20 – Authority Accounts Empty CHECK With No Seeds

Severity: Low

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 608–611

Chain: Solana (Anchor)

Vector Category: Unconstrained Authority

Description

`dao_authority` and `unlock_authority` in `InitializeToken` are declared as `/// CHECK:` with no seeds, no PDA constraints, and no validation. Any pubkey can be passed. These authorities sign transfers from the DAO treasury and unlock vault respectively.

Recommended Fix

Add PDA seed constraints:

```
#[account(seeds = [b"dao-authority"], bump)]
pub dao_authority: AccountInfo<'info>,
#[account(seeds = [b"unlock-authority"], bump)]
pub unlock_authority: AccountInfo<'info>,
```

R-21 – Systemic Missing token::mint Constraints

Severity: Low

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 596–604, 639–640, 645, 678

Chain: Solana (Anchor)

Vector Category: Unconstrained Token Account

Description

Multiple token accounts (`destination` , `dao_treasury` , `unlock_vault`) across `InitializeToken` , `UnlockFunds` , and `DaoTransfer` lack `token::mint = mint` constraints. Any token account for any mint can be passed.

Recommended Fix

Add `token::mint = mint` to all vault and destination accounts.

R-22 – multiplier_bps < 10000 Creates Deflating Schedule

Severity: Informational

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 551–589

Chain: Solana (Anchor)

Vector Category: Configuration Edge Case

Description

If `multiplier_bps < 10000` (e.g., 5000 = 0.5x), prices DECREASE each period. The price gate becomes easier to meet over time instead of harder, inverting the intended escalation behavior.

Recommended Fix

Validate `multiplier_bps >= 10000` during initialization if escalation is always intended.

R-23 – Unbounded period_count – CU Exhaustion

Severity: Low

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 571–586

Chain: Solana (Anchor)

Vector Category: Compute Unit Exhaustion

Description

`compute_escalating_price` iterates `for i in 1..period` at line 571. If `period_count` is very large and a user tries to unlock a late period, the loop consumes excessive compute units, potentially exceeding Solana's 200k CU budget.

Recommended Fix

Use closed-form exponentiation instead of iterative multiplication, or cap `period_count` at initialization:

```
require!(plan.period_count <= 200, CustomError::InvalidPlanConfig);
```

R-24 – secondary_price_multiplier_bps = 0 Collapses Price Gate

Severity: Informational

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 572-576

Chain: Solana (Anchor)

Vector Category: Configuration Edge Case

Description

If `secondary_price_multiplier_bps = 0` and `transition_period > 0`, periods after the transition use `mult = 0`. Line 578: `price = price * 0 / 10000 = 0`. The required price drops to 0 for all post-transition periods, removing the price gate entirely.

Recommended Fix

Validate `secondary_price_multiplier_bps > 0` when `transition_period > 0`.

R-25 – unlock_manager.released Updated After CPI

Severity: Informational

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 269-271

Chain: Solana (Anchor)

Vector Category: CEI Violation

Description

`unlock_manager.released` is incremented at line 271, AFTER the `invoke_signed` CPI at line 269. The transfer hook fires during the CPI and could observe stale `released` value.

Recommended Fix

Move `unlock_manager.released += amount` before the CPI call.

R-26 – decimals Caller-Supplied

Severity: Informational

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 128

Chain: Solana (Anchor)

Vector Category: Incorrect Parameter Source

Description

`decimals` is passed as an instruction argument rather than read from `mint.decimals`. A mismatched `decimals` value causes `transfer_checked` to fail (Token-2022 validates decimals against the mint).

Recommended Fix

Read from mint: `let decimals = ctx.accounts.mint.decimals;`

R-27 – No Minimum Amount Guard

Severity: Informational

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 127, 275

Chain: Solana (Anchor)

Vector Category: Zero-Amount Operation

Description

`unlock_funds` and `dao_transfer` do not validate `amount > 0`. A zero-amount transaction succeeds, consumes gas, and produces misleading logs.

Recommended Fix

```
require!(amount > 0, CustomError::AmountTooSmall);
```

R-28 – dao_treasury_multisig Marked mut Unnecessarily

Severity: Informational

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 685

Chain: Solana (Anchor)

Vector Category: Code Quality

Description

In `DaoTransfer`, `dao_treasury_multisig` is marked `mut` (line 685) but is never written to. The instruction only reads `dao.signers` and `dao.threshold`.

Recommended Fix

Remove `mut` from the account annotation.

R-29 – system_program and rent Unused in DaoTransfer

Severity: Informational

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 687-688

Chain: Solana (Anchor)

Vector Category: Dead Accounts

Description

`DaoTransfer` includes `system_program` and `rent` accounts that are never used in the instruction.

Recommended Fix

Remove unused accounts from the struct.

R-30 – No On-Chain Accounting of DAO Treasury Outflows

Severity: Informational

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 275–338

Chain: Solana (Anchor)

Vector Category: Observability

Description

`dao_transfer` does not track total outflows, per-category spending, or cumulative transfers. There is no on-chain record of how much has been spent from the treasury.

Recommended Fix

Add a `total_disbursed: u64` field to `DaoTreasuryMultisig` and increment on each transfer.

R-31 – Freeze Authority Never Revoked

Severity: Informational

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 492–507, 509–524

Chain: Solana (Anchor)

Vector Category: Admin Override

Description

After minting and freezing the NFT, the `nft_mint_authority` PDA retains freeze authority over the NFT mint. While mint authority is revoked (lines 509–524), freeze authority is not. A future program upgrade could unfreeze and confiscate any soulbound NFT.

Recommended Fix

Revoke freeze authority after freezing:

```
invoke_signed(  
    &set_authority(token_program, &nft_mint, None, AuthorityType::FreezeAc  
    ...,  
)?;
```

R-32 – Permissionless init_nft_registry

Severity: Informational

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 372–376, 714–732

Chain: Solana (Anchor)

Vector Category: Permissionless Initialization

Description

`InitNftRegistry` has no access control. The first caller pays rent for the registry account. The state is initialized to an empty `Vec`, which is the correct initial state regardless of caller.

Impact

Zero – the front-runner gains nothing. They pay rent for an account that needs to exist.

R-33 – Plain Addition in Log Line Causes Overflow Panic

Severity: Informational

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 86

Chain: Solana (Anchor)

Vector Category: Integer Overflow in Log

Description

Line 86: `msg!("{}", owner_amount + dao_amount + unlock_amount)` uses plain `+`. With `overflow-checks = true`, this panics if the sum exceeds `u64::MAX`. While unlikely with real amounts, it's a denial-of-service vector if a malicious caller passes large values.

Recommended Fix

Use `checked_add` or `saturating_add` for the log computation.

R-34 – No Reserved Bytes in State Structs

Severity: Informational

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 815–872

Chain: Solana (Anchor)

Vector Category: Upgrade Compatibility

Description

`DaoTreasuryMultisig`, `UnlockManager`, `NftClaim`, `NftRegistry`, and `UnlockPlan` have no reserved bytes. Future field additions require `realloc`.

Recommended Fix

Add `pub _reserved: [u8; 64]` to each struct.

R-35 – CustomError Enum Missing Required Variants

Severity: Informational

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 875-907

Chain: Solana (Anchor)

Vector Category: Implementation Gap

Description

Recommended mitigations reference error variants that don't exist: `DuplicateSigner`, `DuplicateCategory`, `SelfTransfer`, `InvalidSupply`, `AmountTooSmall`, `CategoryTooLong`, `TooManyPlans`, `InvalidCrankAddress`, `InvalidMint`.

Recommended Fix

Add missing error variants to the `CustomError` enum.

R-36 – rent Sysvar Unnecessary

Severity: Informational

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 616, 632, 688, 812

Chain: Solana (Anchor)

Vector Category: Code Quality

Description

`rent: Sysvar<'info, Rent>` is included in `InitializeToken`, `InitializeUnlockManager`, `DaoTransfer`, and `ClaimNft`. Anchor's `init` macro handles rent automatically since v0.26 — the explicit sysvar is unnecessary overhead.

Recommended Fix

Remove `rent` from all structs that don't use it programmatically.

Whitepaper Discrepancy Findings

WP-01 — NFT Tier Thresholds 10x Lower — Satoshi Guard Missing

Severity: Critical

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 20–21, 395–398

Vector Category: Whitepaper Mismatch

Description

Whitepaper specifies three tiers: Tier 1 Satoshi Guard at 1,500,000 SRC, Tier 2 Community Nominee at 150,000 SRC, Tier 3 Voting Member at 15,000 SRC. Code implements two tiers: code Tier 1 at 150,000 SRC (whitepaper's Tier 2) and code Tier 2 at 15,000 SRC (whitepaper's Tier 3). The highest governance tier (Satoshi Guard) with its 25-NFT cap, mandatory KYC, and 5-year lock is absent.

Impact

Users obtain the highest governance position at 10% of the documented cost. The governance hierarchy's exclusivity is fundamentally undermined.

Recommended Fix

Add `const TIER3_THRESHOLD: u64 = 1_500_000 * 1_000_000_000` and renumber tiers to match the whitepaper (1 = Satoshi Guard, 2 = Community Nominee, 3 = Voting Member).

WP-02 — Tier 3 (Voting Member) Not Implemented

Severity: Critical

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 22-23, 388-391

Vector Category: Missing Feature

Description

Whitepaper's three-tier governance requires "2 out of 3 tiers" for standard proposals and "all 3 tiers" for Satoshi Vault proposals. Code only implements 2 tiers. The Voting Member tier (2,500 NFTs, broadest participation) is missing, making the documented consensus rules architecturally impossible.

Recommended Fix

Add `const NFT_TIER3: u8 = 3`, update `claim_nft` to accept tier 3, add per-tier count tracking.

WP-03 – Transaction Fee 1.29% vs Documented 1%

Severity: High

Confidence: High

File: `programs/transfer_hook/src/lib.rs`

Lines: 152

Vector Category: Specification Mismatch

Description

Whitepaper states "1% transaction fee." Code implements `amount * 129 / 10000 = 1.29%`. Every SRC transfer overcharges by 29 basis points.

Recommended Fix

Align TransferFeeConfig to 100 basis points (1%), or update whitepaper to disclose 1.29%.

WP-04 – One Wallet = One Tier Not Enforced

Severity: High

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 753-758

Vector Category: Anti-Concentration Failure

Description

Whitepaper states "Each wallet may hold only one NFT and belong to only one tier." NftClaim PDA seeds include `tier` byte(`[b"nft-claim", user, &[tier]]`), making it unique per user per tier. A wallet

can claim NFTs in every tier simultaneously.

Recommended Fix

Remove `tier` from PDA seeds: `seeds = [b"nft-claim", user.key().as_ref()]`. Store tier inside the account data.

WP-05 – All NFTs Non-Transferable (Tier 2/3 Should Be Transferable)

Severity: High

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 433–440

Vector Category: Specification Mismatch

Description

Whitepaper: Tier 1 "Non-transferable," Tier 2 and Tier 3 "Transferable." Code applies `initialize_non_transferable_mint` unconditionally for all tiers. This extension is permanent — no way to make a mint transferable after initialization.

Recommended Fix

Conditionally apply NonTransferable only for Tier 1:

```
if tier == NFT_TIER1 {  
    invoke_signed(&initialize_non_transferable_mint(...)?, ...)?;  
}
```

WP-06 – No SRCOIN Lock Mechanism for Governance

Severity: High

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 394–403

Vector Category: Missing Feature

Description

Whitepaper: "Governance rights are conditional on locking" with 5-year/3-year/activation lock durations. "Unlocking results in immediate suspension of governance rights." Code performs a one-time balance

check with no escrow, no lock vault, no duration tracking, no unlock instruction. Users can sell all tokens immediately after claiming a governance NFT.

Recommended Fix

Implement token escrow: create a lock vault PDA per user, escrow required SRC amount, track lock expiration, add `unlock_governance_tokens` instruction that burns/revokes the NFT on unlock.

WP-07 – Per-Tier NFT Caps Not Enforced – Global 100 vs 2,775

Severity: Medium

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 27, 416–419

Vector Category: Scale Limitation

Description

Whitepaper: 25 Tier 1 + 250 Tier 2 + 2,500 Tier 3 = 2,775 total. Code: single global cap of 100 with no per-tier tracking. All 100 slots could go to one tier.

Recommended Fix

Use per-tier counter fields instead of a growing Vec. Add per-tier cap validation.

WP-08 – Undisclosed 40% Fee Burn

Severity: Medium

Confidence: High

File: `programs/transfer_hook/src/lib.rs`

Lines: 247, 275–278

Vector Category: Undisclosed Economic Mechanism

Description

Whitepaper: "All collected transaction fees are routed to a community-managed treasury wallet." Code: 40% of fees are burned, 60% retained in crank wallet. Burn has 21M/year and 420M lifetime caps (lines 21–23). When cap is hit, distribution silently shifts to 0% burn / 100% crank.

Recommended Fix

Update whitepaper to disclose the 40/60 split, burn caps, and mode transition, or modify `collect_fees` to send 100% to treasury.

WP-09 – Allocations Runtime-Configurable

Severity: Medium

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 96-122

Vector Category: Missing Hardcoded Constraints

Description

Whitepaper defines 6 exact categories with fixed amounts and durations. Code accepts arbitrary `Vec<UnlockPlan>` with no validation of category names, amounts, or durations. Only constraint: allocations must sum to `unlock_amount`.

Recommended Fix

Hardcode and validate the six category names, per-year amounts, and duration parameters.

WP-10 – Total Supply Cap Not Validated

Severity: Medium

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 35-86

Vector Category: Missing Invariant

Description

Whitepaper: "Total Supply: 1,000,000,000." Code mints caller-supplied amounts with no sum validation.

Recommended Fix

Add `require!(owner_amount + dao_amount + unlock_amount == TOTAL_SUPPLY)`.

WP-11 – Milestone Evaluation Dates Not Enforced

Severity: Low

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 96-122

Vector Category: Configurable vs Non-Discretionary

Description

Whitepaper: "Milestone conditions and evaluation dates are non-discretionary" — June 9, 2026 and December 9 annually. Code uses admin-configurable `start_ts`, `cliff_ts`, `period_duration`.

Recommended Fix

Hardcode evaluation dates as constants and validate against admin inputs.

WP-12 — Phase One Binary Outcome Not Enforced

Severity: Low

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 124-273

Vector Category: Missing Binary Logic

Description

Whitepaper: 8M SRCOIN "either released or permanently burned" on June 9, 2026. Code uses gradual vesting with no binary checkpoint and no burn instruction. Partial unlocks possible.

Recommended Fix

Add `evaluate_phase_one` instruction with hardcoded date and `burn_expired_allocation` permissionless instruction.

WP-13 — Phase Two Annual Burn Path Missing

Severity: Low

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 124-273

Vector Category: Missing Burn Mechanism

Description

Whitepaper: Annual "binary outcomes: release or permanent burn." No burn instruction exists. Unmet milestones leave tokens idle in vault indefinitely.

Recommended Fix

Add permissionless `burn_expired_period` instruction for community enforcement.

WP-14 – 150M/8M/842M Split Unchecked

Severity: Low

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: 37–39, 50–68

Vector Category: Missing Proportion Validation

Description

Whitepaper: 150M initial + 8M Phase One + 842M Phase Two. Code accepts any proportions.

Recommended Fix

Validate proportions at initialization.

WP-15 – On-Chain Governance Voting Not Implemented

Severity: Informational

Confidence: High

File: `programs/src_token/src/lib.rs`

Lines: N/A

Vector Category: Missing Feature

Description

Whitepaper describes tier-based voting (2-of-3, 3-of-3 consensus). Code has only a flat address-based multisig. No proposals, votes, or tier-weighted quorum.

Recommended Fix

Implement proposal system or clarify off-chain governance in whitepaper.

WP-16 – Tier 3 Annual 1,000 SRCOIN Reward Missing

Severity: Informational
Confidence: High
File: `programs/src_token/src/lib.rs`
Lines: N/A
Vector Category: Missing Feature

Description

Whitepaper: Tier 3 Voting Members have "entitlement to 1,000 SRCOIN per year." No reward instruction exists.

Recommended Fix

Implement `claim_tier3_reward` instruction or clarify off-chain distribution.

Audit Statistics

Metric	Value
Total Findings (Code)	84
Total Findings (Whitepaper)	16
Grand Total	100
Critical	1 (code) + 2 (whitepaper) = 3
High	8 (code) + 4 (whitepaper) = 12
Medium	12 (code) + 4 (whitepaper) = 16
Low	9 (code) + 4 (whitepaper) = 13
Informational	14 (code) + 2 (whitepaper) = 16
Chain	Solana
Framework	Anchor v0.31.0
Token Standard	SPL Token-2022 (TransferHook, TransferFeeConfig, NonTransferable)
Programs Audited	4
Total Lines of Code	2,901
Whitepaper Pages Reviewed	12